

A/B Testing for ML Cheat Sheet

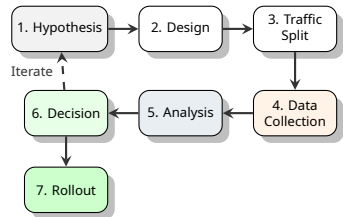
Design, Run, Analyze, and Deploy ML Experiments

1. DEFINITION

A/B Testing (Split Testing): A controlled experiment where two or more variants of a system (e.g., Model A vs. Model B) are shown to users at random to determine which performs better on a specific metric.

- **Why Critical for ML:** Offline metrics (AUC, RMSE) do not always correlate with business value (Revenue, Clicks). A/B testing measures the actual impact on user behavior in production.

2. HIGH LEVEL FLOW



- **Flow:** Hypothesis → Design → Traffic Split → Data Collection → Analysis → Decision → Rollout.

3. EXPERIMENT DESIGN

Concepts

- **Control Group (A):** Users receiving the existing baseline model (or no model).
- **Treatment Group (B):** Users receiving the new experimental model.
- **Randomization:** Ensuring every user has an equal chance of landing in A or B to remove bias.
- **Unit of Randomization:** Who is randomized? (User ID, Session ID, Device ID). Stickiness is key (User should see same model).

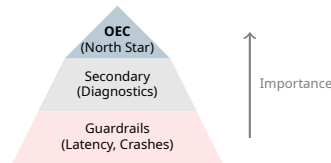
Power Analysis (Sample Size)

Determining how much data is needed to detect a real effect.

```
from statsmodels.stats.power import TTestIndPower

# Estimate Sample Size
analysis = TTestIndPower()
sample_size = analysis.solve_power(
    effect_size=0.1, # Expected improvement
    power=0.8,       # 80% chance to detect
    alpha=0.05,      # 5% false positive rate
)
# Result: N users per group needed
```

4. METRIC SELECTION



- **Primary Metric (OEC):** Overall Evaluation Criterion. The main business goal (e.g., Conversion Rate, Revenue/User).
- **Secondary Metrics:** Diagnostic metrics to understand *why* the primary moved (e.g., CTR, Add-to-Cart rate).
- **Guardrail Metrics:** Operational health metrics that must not degrade (e.g., Latency, Error Rate, Unsubscribes).
- **Offline vs Online:** Offline (Precision/Recall) helps develop the model. Online (Clicks/Revenue) validates business value. They often diverge.

5. TRAFFIC ALLOCATION

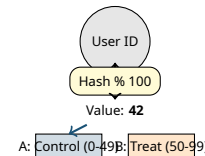
- **Fixed Split:** 50/50 split. Fastest way to statistical significance.
- **Ramp Up:** 1% → 5% → 50%. Safer for risky changes.
- **Multi-Arm Bandit (MAB):** Dynamic allocation. Routes more traffic to the winning model automatically during the test. Balances exploration vs ex-

ploitation.

Deterministic Hashing:

```
import hashlib

def assign_bucket(user_id, salt="exp_v1"):
    # Deterministic assignment
    val = int(hashlib.md5((user_id + salt).encode()).hexdigest(), 16)
    # 50/50 Split
    return "B" if (val % 100) < 50 else "A"
```



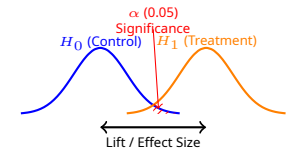
6. DATA COLLECTION & LOGGING

- **Event Logging:** Capture user actions (click, view, purchase).
- **Feature Logging:** Log the exact feature values used at inference time (prevents training-serving skew).
- **Prediction Logging:** Log the model output (score/rank) and the model version ID.
- **Outcome Logging:** Join prediction logs with downstream events (labels) to calculate metrics.

7. STATISTICAL ANALYSIS

Hypothesis Testing

- **Null Hypothesis (H_0):** No difference between A and B.
- **Alternative Hypothesis (H_1):** B is different (or better) than A.
- **p-value:** Probability of seeing the result if H_0 were true. If $p < 0.05$, reject H_0 (Significant).
- **Confidence Interval (CI):** Range where the true difference lies (e.g., $+2\% \pm 0.5\%$). If CI excludes 0, it is significant.



Errors

- **Type I (α):** False Positive. Claiming B is better when it's not.
- **Type II (β):** False Negative. Failing to detect that B is better.

T-Test in Python:

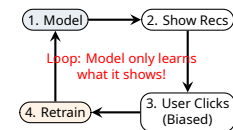
```
from scipy import stats

# Conversion arrays (0 or 1)
control = [0, 1, 0, 0, 1...]
treatment = [0, 1, 1, 0, 1...]

t_stat, p_val = stats.ttest_ind(
    control, treatment
)
if p_val < 0.05:
    print("Significant Difference!")
```

8. ML SPECIFIC CONSIDERATIONS

The Feedback Loop



Mitigating Bias (Epsilon-Greedy):

```
import random

def get_action(model_pred, epsilon=0.1):
    # Exploration vs Exploitation
    if random.random() < epsilon:
        return random.choice(all_actions) # Explore
    return model_pred # Exploit
```

- **Delayed Labels:** Conversion might happen days after the prediction. Metrics lag.
- **Feedback Loops:** The model influences the data it trains on (e.g., Recommender shows items → User clicks → Model learns only from shown items).
- **Exploration Bias:** A deterministic greedy model never learns about

items it doesn't show. Add random exploration (Epsilon-greedy).

- **Model Drift:** Performance degradation over time. A/B test is a snapshot; ongoing monitoring is needed.

9. END TO END EXAMPLE FLOW

Scenario: Recommendation Engine Update.

1. **Baseline:** Collaborative Filtering (Group A).
2. **New Model:** Deep Learning Embedding (Group B).
3. **Setup:** Divert 10% traffic to experiment. Split 50/50 inside that 10%.
4. **Log:** Record recommendations shown and clicks.
5. **Wait:** Run for 2 weeks (capture weekly seasonality).
6. **Analyze:** CTR for A is 2.1%, B is 2.4%. p-value 0.01.
7. **Decision:** B wins. Check guardrails (latency).
8. **Rollout:** Ramp B to 100% of users.

10. PERFORMANCE METRICS

- **Conversion Rate:** $\frac{\text{Conversions}}{\text{Visitors}}$.
- **Lift:** $\frac{\text{Metric}_B - \text{Metric}_A}{\text{Metric}_A}$. Percentage improvement.
- **Statistical Significance:** Confidence that the Lift is real, not noise.
- **Time to Detect:** How long the experiment must run to be valid.
- **Revenue Impact:** Actual \$ gain.
- **Model Accuracy Change:** Did the online accuracy (e.g., Top-1 accuracy) actually improve?

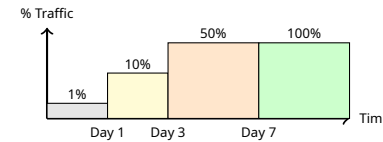
11. DEPLOYMENT AND ROLLOUT

Strategies

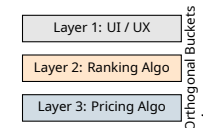
- **Shadow Testing:** Run New Model in parallel but do not show results to user. Log comparison. Low risk.
- **Canary Deployment:** Roll out to small % (e.g., 1%) to check for crashes/latency before full A/B.
- **Rollback Strategy:** Automated revert if Guardrail metrics (Error Rate) spike.

Canary Routing Logic:

```
def route_request(user_id):  
    # Rollout to 5% of users (Canary)  
    if hash(user_id) % 100 < 5:  
        return "model_v2_canary"  
    return "model_v1_baseline"
```



12. SCALING EXPERIMENTS



- **Parallel Experiments:** Running multiple tests simultaneously.
- **Experiment Isolation:** Ensuring Test X (UI color) doesn't bias Test Y (Ranking algo). Use orthogonal hashing/layers.
- **Interaction Effects:** When two changes interact (e.g., New UI + New Algo might perform differently than sum of parts).

13. RELIABILITY AND ETHICS

- **User Fairness:** Does the new model perform worse for specific sub-groups (bias)? Slice analysis required.
- **Bias Detection:** Monitor metrics

across demographics during the test.

- **Risk Mitigation:** Limit exposure of experimental high-risk models.

14. COMMON FAILURE POINTS

- **Sample Ratio Mismatch (SRM):** You designed 50/50 but got 48/52. Indicates a bug in assignment logic. Invalidates test.
- **Peeking:** Checking results early and stopping as soon as it looks significant. Increases Type I error. Fix sample size in advance.
- **Metric Gaming:** Improving CTR (Clickbait) but hurting Retention.
- **Seasonality:** Running a test only on weekends vs weekdays.

15. SUMMARY

Goal: Validate ML impact in the real world scientifically.

- **Process:** Randomize → Measure → Analyze.
- **Mindset:** Trust the online data over offline validation.
- **Outcome:** Data-driven decisions, reduced risk of bad deployments.